# BYTE-LEVEL PREDICTIONS FROM TOKENIZED LANGUAGE MODELS: REMOVING TOKENIZATION BIAS AT INFERENCE TIME

**Anonymous authors**
Paper under double-blind review

## ABSTRACT

Tokenization compresses text and makes language models (LMs) tractable, but it also induces a discrepancy between the distribution a tokenized LM places over tokens and the byte-level distribution of the text it models. We call this discrepancy *tokenization bias*. It is most damaging when a prompt ends in the middle of a token, as in code completion and fill-in-the-middle (FIM), where the greedy tokenization of the prompt can make the correct next character impossible, and it makes models with different vocabularies hard to combine. We show that, for a deterministic tokenizer, the exact probability of any byte string under a tokenized LM is a sum over a set of valid "cover encodings" of that string, and we derive from this a byte-token representation lemma. Based on it, we give an inference-only algorithm that converts next-token predictions into next-byte predictions using a reverse search over cover encodings, with no retraining and no change to the model weights. On synthetic Markov chains the method recovers the true transition probabilities where the token-level model assigns zero probability to valid transitions. On the HumanEval random-span infilling benchmark with CodeLlama-7B, byte-level prediction closes the gap between the two standard prompt templates: in the Suffix-Prefix-Middle template, greedy pass rate rises from 45.0 to 63.9, exceeding the token healing (62.7) and token alignment (63.0) heuristics. Because byte probabilities are vocabulary independent, models with different tokenizers can be ensembled by averaging: top-2 ensembles improve over the best single model on MMLU, PIQA, NQ, TriviaQA and HumanEval, but not on GSM8K, where the two members differ strongly in ability.

## 1 INTRODUCTION

Modern language models do not operate on raw text. A tokenizer, typically based on byte-pair encoding (Gage, 1994; Sennrich et al., 2016) or a related subword scheme (Kudo & Richardson, 2018), maps a byte string to a sequence of tokens from a fixed vocabulary, and the model is trained to predict the next token. This shortens sequences and eases optimization, but it also changes the object being modeled. A tokenized LM defines a distribution over token sequences, whereas users care about the text, that is, about bytes. Tokenization has well-documented side effects, from unequal costs across languages (Petrov et al., 2023) to sensitivity to how the same string is segmented (Kudo, 2018; Cao & Rimell, 2021); see also Mielke et al. (2021); Gastaldi et al. (2024).

In this paper we study one concrete consequence, which we call *tokenization bias*: conditioning a tokenized LM on the single, deterministic tokenization of a prompt does not in general yield the correct conditional distribution over the next *byte*. The problem is easiest to see when a prompt ends mid-token. A deterministic tokenizer such as BPE merges bytes greedily according to its merge rules, so a prompt that ends with the bytes "`a`" is tokenized differently from the same text followed by more characters. The model has only ever seen the merged form of a longer word during training, so after the short form of the prompt it may assign probability zero to the continuation that a user actually wants. The left panel of Figure 1 gives a code-completion example in which the correct completion has zero probability under the token-level model. This situation is common in practice, for example in fill-in-the-middle (FIM) code completion, where the prompt is cut at an arbitrary

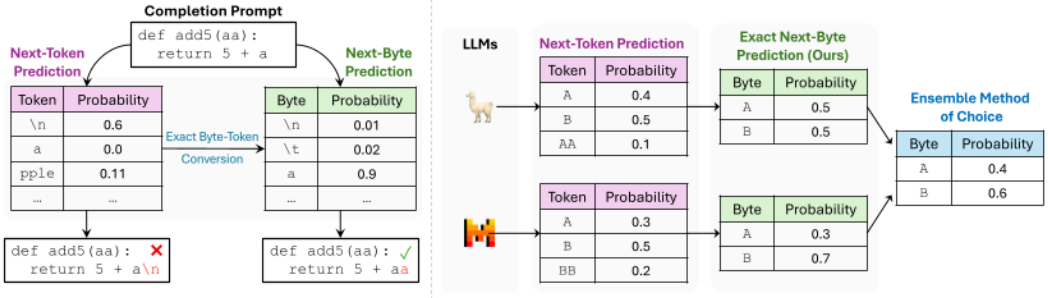



Figure 1: Left: tokenized LMs can experience tokenization bias when prompts end mid-token, as in this code completion example. The correct solution, "a", has zero probability of being chosen at the token level. Our method avoids this problem and predicts the correct byte while using the same model. Right: our method maps next-token predictions of arbitrary tokenized LMs to statistically equivalent next-byte predictions (Section 3). This enables any ensemble strategy, such as averaging or mixtures of experts.

position inside an identifier. A second, related limitation is that the output spaces of models with different vocabularies are disjoint, so their predictions cannot be averaged directly.

Existing remedies are heuristic. Token healing (Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024) back up over the last token(s) of the prompt and constrain generation to stay consistent with the removed characters. These repairs help but are not derived from a statement about what the correct byte-level distribution is. Our starting point is that, if one takes the tokenized LM at face value as a generative model of token sequences and the tokenizer as a deterministic map from token sequences to bytes, then the induced distribution over bytes is well defined, and the bias is an artifact of conditioning on one tokenization of the past instead of marginalizing over all token sequences compatible with the observed bytes.

**Contributions.**

- **Formalization.** We define tokenization bias, *cover encodings* and *valid/invalid encodings*, and prove a byte-token representation lemma (Lemma 1) that expresses the exact probability of a byte string as a sum of token-sequence probabilities over its valid cover encodings.
- **An inference-only algorithm.** We give a reverse (backward tokenization) search that finds all valid cover encodings efficiently, and a sampling procedure that maintains them incrementally, so that any tokenized LM becomes a next-byte predictor without retraining.
- **Applications.** We show (i) exact recovery of transition probabilities on synthetic Markov chains, (ii) that byte-level predictions remove most of the degradation of token-level models on FIM prompts that end mid-token, outperforming token healing and token alignment on a Python infilling benchmark, and (iii) that, since byte probabilities are vocabulary independent, models with different tokenizers can be ensembled by simple averaging, with gains on most benchmarks and a clear failure case that we analyze.

## 2 Related Work

**Subword tokenization and its side effects.** Subword vocabularies are built by BPE (Gage, 1994; Sennrich et al., 2016) or unigram models (Kudo, 2018), and are commonly implemented in libraries such as SentencePiece (Kudo & Richardson, 2018); Mielke et al. (2021) survey open-vocabulary modeling. A string generally admits many token sequences. Subword regularization (Kudo, 2018) and BPE-dropout (Provilkov et al., 2020) exploit this by sampling alternative segmentations during training, and Cao & Rimell (2021) argue that LMs should be evaluated by marginal likelihood over tokenizations. In contrast, we marginalize at inference time, for a model trained with the canonical (deterministic) tokenization, to obtain next-*byte* probabilities, and use the determinism

of the tokenizer to restrict the sum to valid encodings. Pimentel & Meister (2024) study how to compute the probability of a word from a subword LM, a question closely related to the prompt-boundary issue studied here. Gastaldi et al. (2024) give a formal treatment of tokenizers as maps between strings and token sequences, and Petrov et al. (2023) document disparities that tokenizers introduce across languages.

**Prompt-boundary heuristics.** Token healing (Dagan et al., 2024) removes the last token of the prompt and constrains the next token to start with the removed characters, while token alignment (Athiwaratkun et al., 2024) matches characters between the prompt and the vocabulary to complete subwords. Both address the same mid-token failure mode that we study and serve as our main baselines for FIM. They operate on a single (partially repaired) token history, whereas our method accounts for all valid token histories consistent with the bytes.

**Fill-in-the-middle.** FIM training (Bavarian et al., 2022; Fried et al., 2023) teaches left-to-right LMs to infill by rearranging a document into prefix, suffix and middle. Two standard prompt layouts are Prefix-Suffix-Middle (PSM) and Suffix-Prefix-Middle (SPM), which are supported by Code Llama (Rozière et al., 2023). We evaluate both on the HumanEval random-span infilling benchmark of Bavarian et al. (2022), derived from HumanEval (Chen et al., 2021).

**Token-free and byte-level models.** One way to avoid tokenization bias is to avoid tokens altogether, using character- or byte-level architectures (Clark et al., 2022; Tay et al., 2022; Xue et al., 2022; Yu et al., 2023; Wang et al., 2024). These require training new models and handling much longer sequences. Our approach instead converts an already trained tokenized LM into a byte-level predictor at inference time.

**Ensembling language models with different vocabularies.** Averaging predictions of several models is a classic way to improve accuracy (e.g., products and mixtures of experts, Hinton, 2002; Jacobs et al., 1991; Shazeer et al., 2017), but it requires a shared output space. For LLMs with different vocabularies, prior work aligns or projects vocabularies (Xu et al., 2024; Huang et al., 2024) or fuses knowledge across models (Wan et al., 2024). Byte probabilities provide a common output space without any alignment between vocabularies, because every tokenizer decodes to bytes.

# 3 METHODOLOGY

## 3.1 SETTING AND TOKENIZATION BIAS

Let $\mathcal{A}$ be the set of bytes and $\mathcal{V}$ a token vocabulary, where every token $v \in \mathcal{V}$ has a byte-string value $\mathrm{decode}(v) \in \mathcal{A}^+$. Extending $\mathrm{decode}$ by concatenation to token sequences, a deterministic tokenizer is a map $\mathrm{encode} : \mathcal{A}^* \to \mathcal{V}^*$ satisfying $\mathrm{decode}(\mathrm{encode}(x)) = x$. We write $x_1^n = (x_1, \ldots, x_n)$ for a byte string and $\vec{t} = (t_1, \ldots, t_k)$ for a token sequence, and let $\ell(\vec{t}) = |\mathrm{decode}(\vec{t})|$ be its length in bytes. Let $L$ denote the maximum byte length of a token. A tokenized LM is an autoregressive model $P(t_{i+1} \mid t_1^i)$ over tokens, with sequence probability $P(\vec{t}) = \prod_{i=1}^{k} P(t_i \mid t_1^{i-1})$. It is trained on text tokenized with $\mathrm{encode}$, so we treat it as a generative model of *canonical* token sequences, those $\vec{t}$ with $\mathrm{encode}(\mathrm{decode}(\vec{t})) = \vec{t}$.

A tokenized LM is usually used at the byte level as follows: to continue a prompt $x_1^n$, tokenize it as $\vec{t} = \mathrm{encode}(x_1^n)$ and sample the next token from $P(\cdot \mid \vec{t})$. This amounts to the estimator

$$\widehat{P}(x_{n+1} = b \mid x_1^n) \;=\; \sum_{v \in \mathcal{V}:\ \mathrm{decode}(v)_1 = b} P(v \mid \mathrm{encode}(x_1^n)), \tag{1}$$

where $\mathrm{decode}(v)_1$ is the first byte of $v$. The target is instead the conditional of the byte process induced by the LM, $P(x_{n+1} \mid x_1^n) = P(x_1^{n+1})/P(x_1^n)$, with $P(x_1^n)$ the probability that the decoded output of the LM starts with $x_1^n$.

**Definition 1** (Tokenization bias). *A tokenized LM exhibits tokenization bias on $x_1^n$ if $\widehat{P}(\cdot \mid x_1^n) \neq P(\cdot \mid x_1^n)$, i.e., if conditioning on the single tokenization $\mathrm{encode}(x_1^n)$ does not reproduce the byte-level conditional.*

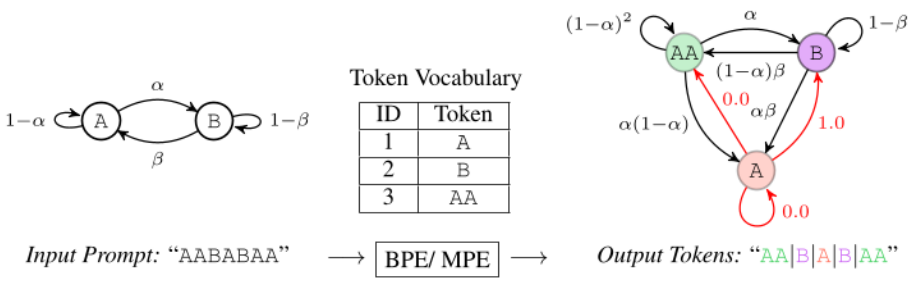



Figure 2: Tokenization bias on a 1st-order Markov chain. Given the context token "A", the model never samples the next token "A" (red, probability 0.0), rather than with probability $1 - \alpha$. In practice, this bias occurs when prompts end with tokens that are part of another token, a common issue in FIM tasks, leading to incorrect completions by the model.

**Example.** Consider a Markov chain over bytes $\{\texttt{A}, \texttt{B}\}$ with $P(\texttt{A} \mid \texttt{A}) = 1 - \alpha$, and a BPE vocabulary $\{\texttt{A}, \texttt{B}, \texttt{AA}\}$ (Figure 2). Given a prompt ending in the single byte A, the tokenizer would have produced AA had another A followed, so after the token A the model learns that neither A nor AA can follow. Equation equation 1 thus gives probability 0 to the next byte A, while the true value is $1 - \alpha$.

### 3.2 Cover encodings and the byte-token representation lemma

The bias arises because Eq. equation 1 uses only one token history. To compute $P(x_1^n)$ exactly we sum over all the token sequences that can generate $x_1^n$ as a prefix of their output.

**Definition 2** (Cover encodings)**.** *The set of* cover encodings *of* $x_1^n$ *is*

$$\text{cover}(x_1^n) = \left\{\vec{t} = (t_1, \ldots, t_k) \;:\; \text{decode}(\vec{t})_{1:n} = x_1^n,\ \ell(t_1^{k-1}) < n\right\},$$

*that is, the token sequences whose decoded output starts with* $x_1^n$ *and whose last token is the first one that reaches or passes byte* $n$*.*

By minimality of the last token, no cover encoding is a prefix of another one. In Figure 3 (left), the query string yummy has cover encodings that end exactly at byte 5 and others whose last token extends beyond it.

**Definition 3** (Valid and invalid encodings)**.** *A token sequence* $\vec{t}$ *is a* valid encoding *if* $\text{encode}(\text{decode}(\vec{t})) = \vec{t}$*, and an* invalid encoding *otherwise.*

For a deterministic tokenizer, each string has exactly one valid encoding (the canonical one), and any other token sequence that decodes to the same string is invalid, e.g., $(\texttt{y}, \texttt{u}, \texttt{m}, \texttt{m}, \texttt{y}, \texttt{u})$ in Figure 3 (left) if the tokenizer would merge yu. We make the following idealizing assumption, which formalizes "statistically equivalent byte-level process".

**Assumption 1.** *(i) The LM assigns probability zero to invalid encodings, so that it is a distribution over canonical token sequences. (ii) Every prefix of a valid encoding is a valid encoding (prefix-closure of validity).*

Assumption 1(i) holds for an LM that perfectly models canonical data; for real models, we simply discard invalid encodings, which amounts to renormalizing. Assumption 1(ii) is used only to make the search of Section 3.3 efficient (Lemma 1 itself does not need it), and is natural for BPE, where merges are never undone.

**Lemma 1** (Byte-token representation)**.** *Under Assumption 1(i), for every byte string* $x_1^n$*,*

$$P(x_1^n) \;=\; \sum_{\vec{t} \in \text{cover}(x_1^n),\ \vec{t}\ \textit{valid}} P(\vec{t}). \tag{2}$$

*Consequently,* $P(x_{n+1} \mid x_1^n) = P(x_1^{n+1})/P(x_1^n)$ *is computable from token-level conditionals alone.*

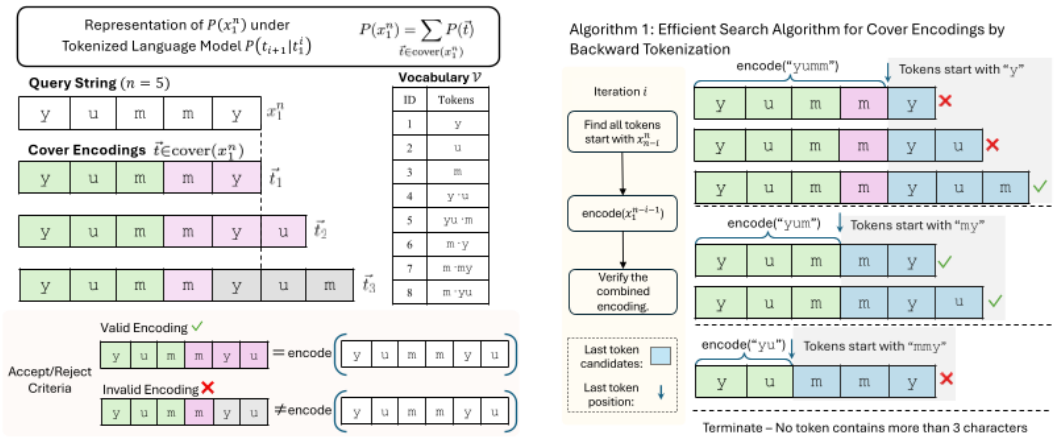



Figure 3: Left: representation of $P(x_1^n)$ using a tokenized LM, with an example of cover encodings and valid/invalid encodings. Right: illustration of Algorithm 1 for the cover encodings search. Green ticks and red crosses denote valid and invalid encodings respectively (Definition 3). The termination step can be easily implemented despite not shown in the algorithm. We use BPE in this example.

**Algorithm 1** Cover encodings by backward tokenization

**Require:** byte string $x_1^n$, tokenizer encode, vocabulary $\mathcal{V}$ with maximum token length $L$

$\mathcal{C} \leftarrow \emptyset$
**for** $i = 0, 1, \ldots, \min(L, n) - 1$ **do**
  $s \leftarrow n - i$; $\vec{p} \leftarrow \text{encode}(x_1^{s-1})$
  **for all** $v \in \mathcal{V}$ such that $\text{decode}(v)$ starts with $x_s^n$ **do**
    $\vec{t} \leftarrow \vec{p} \oplus (v)$
    **if** $\text{encode}(\text{decode}(\vec{t})) = \vec{t}$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \{\vec{t}\}$
    **end if**
  **end for**
**end for**
**return** $\mathcal{C}$ $\quad (= \{\vec{t} \in \text{cover}(x_1^n) \text{ valid}\})$

A proof is given in Appendix A. The idea is that the event "the decoded output starts with $x_1^n$" is the disjoint union, over cover encodings $\vec{t}$, of the events "the token sequence starts with $\vec{t}$", and invalid encodings have probability zero. Comparing with Eq. equation 1, the token-level estimator keeps only the cover encodings that end exactly at byte $n$ and that equal the canonical $\text{encode}(x_1^n)$. It drops the cover encodings whose last token extends beyond byte $n$ (such as `AA` for the prompt `A`), which is the source of the bias.

### 3.3 Efficient search for cover encodings

Enumerating all token sequences that decode to a prefix of the context is exponential. Two observations make the search cheap. First, a cover encoding is determined by the byte position $s \leq n$ at which its last token starts and by that token. Second, if the last token starts at $s$, the preceding tokens decode to $x_1^{s-1}$ and, by Assumption 1(ii), a valid encoding must have a valid prefix, which is unique and equals $\text{encode}(x_1^{s-1})$. We therefore search backwards from the end of the string (Algorithm 1 and Figure 3, right). At iteration $i = 0, 1, \ldots$, we set $s = n - i$ and gather all vocabulary tokens whose byte string starts with $x_s^n$ (a trie lookup). We then form the candidate $\text{encode}(x_1^{s-1}) \oplus v$ for each such token $v$ and keep it if it is valid, i.e., if $\text{encode}(\text{decode}(\cdot))$ reproduces it. Since no token is longer than $L$ bytes, the search stops after at most $L$ iterations ("no token contains more than 3 characters" in the toy example of Figure 3).

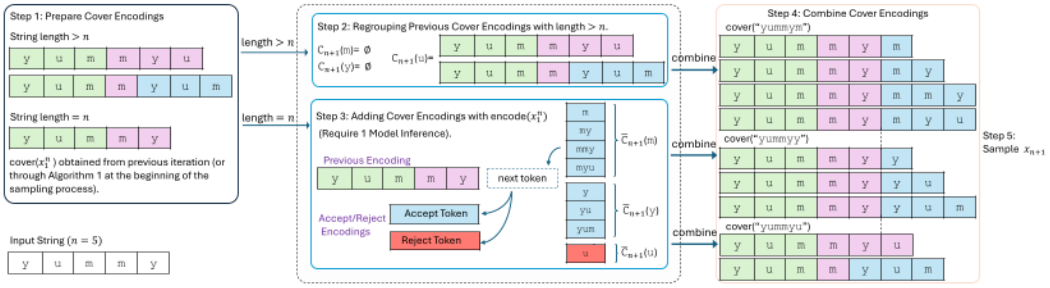



Figure 4: Illustration of the sampling process for $x_{n+1}$ from a tokenized LM $P(t_{k+1} \mid t_1^k)$, with the example following Figure 3. We do not include $P(\vec{t})$ for simplicity. Details in Section 3.4.

### 3.4 Sampling the next byte

Algorithm 1 is used once on the prompt. Afterwards, the set of valid cover encodings is maintained incrementally while bytes are generated (Figure 4). Let $\mathcal{C}_n$ be the valid cover encodings of the current string $x_1^n$, each stored together with its probability $P(\vec{t})$. For every candidate next byte $b$, define the mass

$$M(b) \;=\; \sum_{\substack{\vec{t}\in\mathcal{C}_n,\ \ell(\vec{t})>n \\ \mathrm{decode}(\vec{t})_{n+1}=b}} P(\vec{t}) \;+\; \sum_{\vec{t}\in\mathcal{C}_n,\ \ell(\vec{t})=n} \;\sum_{\substack{v\in\mathcal{V}:\ \mathrm{decode}(v)_1=b \\ \vec{t}\oplus(v)\ \text{valid}}} P(\vec{t})\,P(v \mid \vec{t}). \tag{3}$$

The first term regroups the previous cover encodings that already extend beyond byte $n$ by their $(n{+}1)$-th byte (Steps 1–2 in Figure 4), and needs no model call. The second term extends the encodings that end exactly at byte $n$ with each next token that starts with $b$, accepting the extension only if it is a valid encoding (Step 3); it requires one forward pass of the LM per such encoding. For a fixed start position $s$ there is at most one cover encoding ending exactly at byte $n$, so there are at most $L$ of them and hence at most $L$ forward passes per generated byte without any caching. By Lemma 1, $M(b) = P(x_1^n b)$ and

$$P(x_{n+1} = b \mid x_1^n) \;=\; \frac{M(b)}{\sum_{b'} M(b')}. \tag{4}$$

The normalizer equals $P(x_1^n)$ when Assumption 1(i) holds exactly; normalizing by $\sum_{b'} M(b')$ handles the practical case where the LM leaks some mass to invalid encodings. We then sample (or take the argmax of) $x_{n+1}$ (Step 5), keep the encodings of $\mathcal{C}_n$ extended consistently with $b$ as $\mathcal{C}_{n+1}$ (Step 4), and repeat. Because no invalid encoding ever enters the set, the procedure is exact for any LM satisfying Assumption 1(i), and requires no weight updates. Beam search over bytes is obtained by keeping one such state per hypothesis and scoring hypotheses by the sum of byte log-probabilities from Eq. equation 4.

### 3.5 Applications: FIM and cross-vocabulary ensembling

**Fill-in-the-middle.** In the SPM template the prompt ends with the prefix, so generation of the middle starts from an arbitrary byte position that may fall inside a token; in PSM the middle follows a separate marker, so tokenization problems are much milder. The byte-level sampler above conditions on the exact prompt bytes and does not rely on the prompt's tokenization being compatible with the continuation.

**Ensembling.** Given models $M_1, \ldots, M_m$ with arbitrary (possibly different) tokenizers, each model maintains its own cover-encoding state and produces a distribution $P_j(\cdot \mid x_1^n)$ over the 256 possible next bytes via Eq. equation 4. We combine them by uniform averaging, $\bar{P}(b \mid x_1^n) = \frac{1}{m}\sum_j P_j(b \mid x_1^n)$, sample or select the next byte from $\bar{P}$, and append it to the context of every model (Figure 1, right). Other combination rules (e.g., weighted mixtures or products of experts) could be applied in the same byte space; we only evaluate uniform averaging.

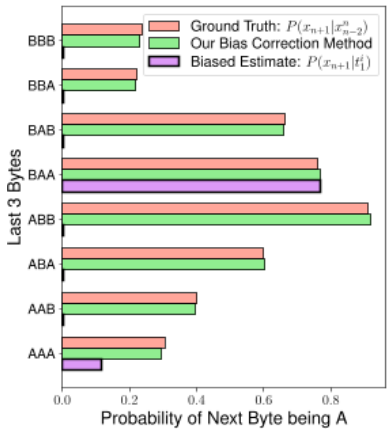



Figure 5: Tokenization bias on a 3rd-order Markov chain. Our byte-token conversion (bias correction) method of Section 3 accurately recovers $P(x_{n+1} \mid x_1^n)$ of the original chain, while the biased token-level estimate does not.

# 4 EXPERIMENTS

## 4.1 SETUP

**Models.** We use general-purpose models Llama2-7B (Touvron et al., 2023), Yi-1.5-6B (Young et al., 2024) and Mistral-7B-v0.3 (Jiang et al., 2023), and code models CodeLlama-7B and CodeLlama-13B (Rozière et al., 2023) and Yi-Coder-1.5B. All are Transformer LLMs (Vaswani et al., 2017). For synthetic experiments we use 1st-order and 3rd-order Markov chains and a 6-layer GPT-2-style model (Radford et al., 2019) trained on synthetic data. All results use the pretrained models without any additional training; the byte-level method is applied at inference time only.

**Tasks and metrics.** (i) *Synthetic analysis*: recovery of Markov chain transition probabilities. (ii) *Code infilling*: the HumanEval Random Span benchmark (Bavarian et al., 2022) on Python with CodeLlama-7B, with both PSM and SPM templates, reporting pass rate under greedy decoding and pass@10 (maximum over a temperature sweep, cf. Figure 6). (iii) *Knowledge and reasoning*: MMLU (Hendrycks et al., 2021), PIQA (Bisk et al., 2020), Natural Questions (NQ, Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017) and GSM8K (Cobbe et al., 2021), reporting accuracy computed from token-level versus byte-level probabilities. (iv) *Code generation*: HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), pass@1.

**Baselines.** *Token-level* is the standard autoregressive prediction of each model. For FIM in SPM mode we also compare to *token healing* (Dagan et al., 2024) and *token alignment* (Athiwaratkun et al., 2024); neither is applied in PSM mode. For ensembles we compare against a *voting* baseline over the top-2 models. Ensembles use the two best-performing individual models and uniform averaging in byte space. Every number below is a single reported value; we have no variance estimates, so differences of a few tenths of a point should not be over-interpreted.

## 4.2 VERIFYING TOKENIZATION BIAS ON SYNTHETIC DATA

On the 1st-order Markov chain of Figure 2, token-level prediction assigns probability $0.0$ to the next token being `A` after the context token `A`, whereas the ground truth is $1 - \alpha$. In the 3rd-order experiment (Figure 5), the trained tokenized model shows severe bias in its predictive distribution compared to the ground truth: the biased estimate of the next-byte probability is visibly off for most contexts, and applying our byte-token conversion recovers the original transition probabilities $P(x_{n+1} \mid x_{n-2}^n)$ closely in all eight contexts.

Table 1: Pass rates (%) on the HumanEval Random Span infilling benchmark with CodeLlama-7B. @10 is the maximum over a temperature sweep. Healing and alignment are only applied in SPM mode.

| | PSM | | SPM | |
|---|---|---|---|---|
| Method | Greedy | @10 | Greedy | @10 |
| Token-level | 64.9 | 84.0 | 45.0 | 66.5 |
| + token healing | – | – | 62.7 | 83.8 |
| + token alignment | – | – | 63.0 | 82.9 |
| Byte-level (ours) | **65.7** | **84.1** | **63.9** | **84.3** |

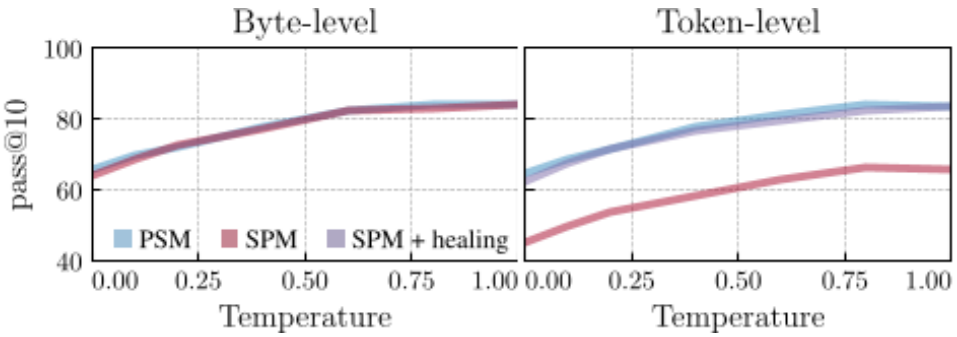



Figure 6: pass@10 on the HumanEval Random Span benchmark (Bavarian et al., 2022) with CodeLlama-7B as a function of sampling temperature, for PSM, SPM and SPM with token healing (Dagan et al., 2024). With byte-level predictions (left), the three curves essentially coincide; with token-level predictions (right), SPM is far below PSM at all temperatures shown, and healing recovers most of the gap. The SPM template forces the model to predict from the middle of a token, which byte-level predictions handle.

### 4.3 Fill-in-the-middle code completion

Table 1 and Figure 6 show results on the HumanEval Random Span benchmark with CodeLlama-7B. Byte-level prediction gives the best result in every column.

**Token-level SPM degradation.** Token-level models suffer a large drop when switching from PSM to SPM: greedy pass rate falls from 64.9 to 45.0 (pass@10 from 84.0 to 66.5). This is consistent with prompts that end mid-token being out-of-distribution for the tokenized model.

**Byte-level recovery.** Byte-level prediction recovers the SPM greedy score to 63.9, close to the PSM score, an improvement of 18.9 points (about 42% relative) over the token-level SPM baseline. It outperforms the heuristics: token healing reaches 62.7 greedy and 83.8 @10, and token alignment 63.0 and 82.9, compared with 63.9 and 84.3 for ours. The margins over the heuristics are small (0.9 and 1.2 points greedy, 0.5 and 1.4 points @10), and without variance estimates we cannot state that they are statistically reliable; the large gain over the unrepaired token-level baseline is not in doubt.

**Unexpected gains in PSM.** Even in PSM, where tokenization issues are expected to be minimal, byte-level greedy generation is slightly better than token-level greedy (65.7 vs. 64.9), with @10 essentially equal (84.1 vs. 84.0). A small gain of this size could be within noise; we do not have the repeated runs needed to tell.

**Failure cases.** We analyzed individual cases in which the prompt ends in the middle of a word, such as the identifiers `delimeter`, `collatz` and `palindrome`. The token-level model failed to complete the word correctly because of token-boundary mismatches, whereas the byte-level method completed it correctly.

Table 2: Accuracy (%) of open-source LMs on knowledge and reasoning benchmarks with token-level (T) and byte-level (B) probabilities. Voting and ensembles use the top-2 performing models; "–" means not available. Best byte-level result per benchmark in bold.

| | MMLU | | PIQA | | NQ | | TriviaQA | | GSM8K | |
|---|---|---|---|---|---|---|---|---|---|---|
| Model | T | B | T | B | T | B | T | B | T | B |
| Llama2-7B | 45.4 | 45.7 | 78.1 | 78.2 | 25.0 | 23.4 | 58.4 | 58.1 | 15.1 | 12.5 |
| Yi-1.5-6B | 63.4 | 63.4 | 78.5 | 78.5 | 22.8 | 22.7 | 53.7 | 53.4 | 61.3 | **61.5** |
| Mistral-7B-v0.3 | 62.1 | 62.1 | 80.1 | 80.3 | 28.5 | 28.8 | 63.6 | 63.5 | 39.0 | 39.0 |
| Voting (top-2) | – | 62.1 | – | 80.1 | – | – | – | – | – | – |
| Top-2 ensemble (ours) | – | **65.4** | – | **80.7** | – | **30.0** | – | **64.2** | – | 55.8 |

Table 3: Pass@1 (%) of code LMs on HumanEval and MBPP with token-level (T) and byte-level (B) probabilities. The ensemble on HumanEval combines Yi-Coder-1.5B and CodeLlama-13B; "–" means not available.

| | HumanEval @1 | | MBPP @1 | |
|---|---|---|---|---|
| Model | T | B | T | B |
| CodeLlama-7B | 32.3 | 28.7 | 40.6 | 42.4 |
| CodeLlama-13B | 35.8 | 33.5 | 47.6 | 47.4 |
| Yi-Coder-1.5B | 38.4 | 36.5 | 52.8 | **53.6** |
| Top-2 ensemble (ours) | – | **42.1** | – | **53.6** |

### 4.4 General and code language models; ensembling

Table 2 compares token-level and byte-level evaluation of three general-purpose models, and of the top-2 byte-level ensemble.

**Byte-level vs. token-level evaluation.** Converting a model to byte-level predictions leaves accuracy essentially unchanged in most cells (differences of at most 0.3 points on MMLU, PIQA, TriviaQA for all models, and on NQ for Yi and Mistral). The exceptions are Llama2-7B on NQ (25.0 to 23.4) and on GSM8K (15.1 to 12.5), which are drops of 1.6 and 2.6 points; on GSM8K, Yi-1.5-6B improves slightly (61.3 to 61.5). We do not have an experiment that isolates the cause of the Llama2-7B drops. Since the two decoding rules are different procedures (byte-level greedy selection marginalizes over tokenizations, while token-level greedy selects the single most likely token), they need not agree even for an exact conversion.

**Ensembling across vocabularies.** The three models above use different vocabularies; mapping each to byte space lets us average them directly. The top-2 ensemble outperforms the best individual model on MMLU (65.4 vs. 63.4, $+2.0$), PIQA (80.7 vs. 80.3 for the best byte-level and 80.1 for the best token-level; $+0.4$ and $+0.6$), NQ (30.0 vs. 28.8, $+1.2$) and TriviaQA (64.2 vs. 63.6, $+0.6$). The voting baseline reaches 62.1 on MMLU and 80.1 on PIQA, matching one member and below the ensemble in both cases. The gains on PIQA and TriviaQA are small enough to be within plausible run-to-run noise.

**Limitation: divergent members.** Ensembling does not help when the members differ strongly in ability. On GSM8K, Yi-1.5-6B (61.5 byte-level) is far better than Mistral-7B-v0.3 (39.0), and the ensemble reaches only 55.8, which is below the better member. Uniform averaging gives the weaker model as much influence as the stronger one, so this is the behavior one expects from an unweighted combination, though we did not test weighted mixtures or other aggregation rules.

**Code generation.** Table 3 shows code LMs on HumanEval and MBPP. Here, byte-level evaluation of the *single* models is not uniformly better: on HumanEval, it is lower for all three models (by

Table 4: Ablation on beam width for byte-level prediction in the FIM task (pass rate, %, on HumanEval).

| Beam width | PSM | SPM |
|---|---|---|
| 1 | 65.7 | 63.9 |
| 2 | 66.8 | 65.9 |
| 4 | 67.7 | 66.8 |

3.6, 2.3 and 1.9 points), while on MBPP it is higher for CodeLlama-7B (+1.8) and Yi-Coder-1.5B (+0.8) and about equal for CodeLlama-13B ($-0.2$). We do not have an analysis explaining the HumanEval drops; as above, a difference between byte-level and token-level decoding rules is a possible factor that we did not test. The ensemble of Yi-Coder-1.5B and CodeLlama-13B reaches 42.1 on HumanEval, $+3.7$ over the best individual token-level model (38.4) and $+5.6$ over the best byte-level one (36.5). On MBPP the ensemble (53.6) ties the best single byte-level model, Yi-Coder-1.5B, rather than improving on it.

### 4.5 Ablation: beam width

Table 4 reports byte-level beam search with widths 1, 2 and 4 on the FIM task. The width-1 values coincide with the byte-level greedy results in Table 1. Pass rates increase monotonically with the beam width in both templates, by up to 2.0 points in PSM (65.7 to 67.7) and 2.9 points in SPM (63.9 to 66.8). The PSM–SPM gap shrinks from 1.8 to 0.9 points. We tested only three widths and did not measure the runtime cost of larger beams.

## 5 Conclusion

We identified tokenization bias, the mismatch between the next-token distribution of a tokenized LM conditioned on a single tokenization and the next-byte distribution of the process it models. We showed that the exact byte-level probability of a string is a sum over valid cover encodings, and turned this into an inference-only algorithm with a reverse search and incremental sampling that converts any tokenized LM into a next-byte predictor without retraining. Empirically, the method recovers transition probabilities on synthetic Markov chains, removes most of the SPM-template degradation on code infilling (greedy pass rate 45.0 to 63.9, versus 62.7 and 63.0 for token healing and alignment), and enables averaging of models with different vocabularies, with gains on most benchmarks.

**Limitations.** (i) Exactness relies on Assumption 1: real LMs may put mass on invalid encodings, which we discard and renormalize, and the efficient search additionally uses the prefix-closure of validity, which we state as an assumption rather than prove for every tokenizer. (ii) Each generated byte needs up to $L$ forward passes in the worst case (Section 3.4); we did not measure wall-clock overhead or the effect of caching. (iii) Byte-level evaluation of individual models is not uniformly better than token-level evaluation: it lowers Llama2-7B on NQ and GSM8K and all code models on HumanEval, for reasons we did not isolate. (iv) Ensembling by uniform averaging fails when members differ greatly in quality (GSM8K), and we evaluated only the top-2 ensembles with this rule. (v) All results are single reported values without confidence intervals, so small differences are not conclusive; the baselines' margins on FIM are small compared to the large gain over the unrepaired token-level baseline. Future work includes weighted or learned byte-level mixtures, ensembles of more than two models, and measuring and reducing the computational cost of the cover-encoding search.

## References

Ben Athiwaratkun, Shiqi Wang, Mingyue Shang, Yuchen Tian, Zijian Wang, et al. Token alignment via character matching for subword completion. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7432–7439, 2020.

Kris Cao and Laura Rimell. You should evaluate your language model on marginal likelihood over tokenisations. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 2104–2114, 2021.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. Getting the most out of your tokenizer for pre-training and domain adaptation. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. InCoder: A generative model for code infilling and synthesis. In *International Conference on Learning Representations*, 2023.

Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.

Juan Luis Gastaldi, John Terilla, Luca Malagutti, Brian DuSell, Tim Vieira, and Ryan Cotterell. The foundations of tokenization: Statistical and computational concerns. *arXiv preprint arXiv:2407.11606*, 2024.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002.

Yichong Huang, Xiaocheng Feng, Baohang Li, Yang Xiang, Hui Wang, Bing Qin, and Ting Liu. Ensemble learning for heterogeneous large language models with deep parallel collaboration. *arXiv preprint arXiv:2404.12715*, 2024.

Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.

Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pp. 1601–1611, 2017.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, pp. 66–75, 2018.

Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 66–71, 2018.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, et al. Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. *arXiv preprint arXiv:2112.10508*, 2021.

Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems*, 2023.

Tiago Pimentel and Clara Meister. How to compute the probability of a word. *arXiv preprint arXiv:2406.14561*, 2024.

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. BPE-dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1882–1892, 2020.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pp. 1715–1725, 2016.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations*, 2022.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. Knowledge fusion of large language models. In *International Conference on Learning Representations*, 2024.

Junxiong Wang, Tushaar Gangavarapu, Jing Nathan Yan, and Alexander M. Rush. MambaByte: Token-free selective state space model. *arXiv preprint arXiv:2401.13660*, 2024.

Yangyifan Xu, Jinliang Lu, and Jiajun Zhang. Bridging the gap between different vocabularies for LLM ensemble. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, 2024.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.

Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, et al. Yi: Open foundation models by 01.AI. *arXiv preprint arXiv:2403.04652*, 2024.

Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Advances in Neural Information Processing Systems*, 2023.

## A Proof of Lemma 1

Let $T = (T_1, T_2, \dots)$ be a random token sequence drawn from the LM, and let $E_n$ be the event that $\mathrm{decode}(T)$ starts with $x_1^n$, so that $P(x_1^n) = \Pr(E_n)$. On $E_n$, let $k^* = \min\{k : \ell(T_1^k) \geq n\}$; then $T_1^{k^*} \in \mathrm{cover}(x_1^n)$ by Definition 2. Conversely, if $T$ starts with some $\vec{t} \in \mathrm{cover}(x_1^n)$, then $\mathrm{decode}(T)$ starts with $\mathrm{decode}(\vec{t})$, which starts with $x_1^n$, so $E_n$ occurs. Hence

$$E_n = \bigcup_{\vec{t} \in \mathrm{cover}(x_1^n)} \{T \text{ starts with } \vec{t}\}.$$

The union is disjoint: if $T$ starts with two distinct cover encodings $\vec{t} \neq \vec{t}'$, one would be a proper prefix of the other, but then the shorter one would already satisfy $\ell \geq n$, contradicting $\ell(t_1^{k-1}) < n$ for the longer one. Therefore $P(x_1^n) = \sum_{\vec{t} \in \mathrm{cover}(x_1^n)} \Pr(T \text{ starts with } \vec{t}) = \sum_{\vec{t} \in \mathrm{cover}(x_1^n)} P(\vec{t})$, where the last step uses the chain rule $P(\vec{t}) = \prod_i P(t_i \mid t_1^{i-1})$. By Assumption 1(i), terms with invalid $\vec{t}$ are zero, which yields Eq. equation 2. The expression for $P(x_{n+1} \mid x_1^n)$ follows from the definition of conditional probability. □

**Check on the Markov example.** For the vocabulary $\{\mathtt{A}, \mathtt{B}, \mathtt{AA}\}$ and $x_1^1 = \mathtt{A}$, the cover encodings are $(\mathtt{A})$ and $(\mathtt{AA})$, both valid. For $x_1^2 = \mathtt{AA}$, the cover encodings are $(\mathtt{AA})$, $(\mathtt{A}, \mathtt{A})$ and $(\mathtt{A}, \mathtt{AA})$; the latter two are invalid because $\mathrm{encode}(\mathtt{AA}) = (\mathtt{AA}) \neq (\mathtt{A}, \mathtt{A})$ and $\mathrm{encode}(\mathtt{AAA}) \neq (\mathtt{A}, \mathtt{AA})$ (the tokenizer merges $\mathtt{AA}$ first). Hence $P(\mathtt{AA}) = P((\mathtt{AA}))$ and $P(x_2 = \mathtt{A} \mid x_1 = \mathtt{A}) = P((\mathtt{AA}))/\big(P((\mathtt{A})) + P((\mathtt{AA}))\big)$, which is positive, in contrast to the token-level estimate of Eq. equation 1, which is 0.